

Синхронизация и споделено състояние

Конкурентното програмиране разглежда изпълнението на програми, в които няколко нишки или задачи могат да напредват едновременно или привидно едновременно. Това позволява по-добро използване на процесорното време, по-висока отзивчивост и възможност за обработка на множество операции в припокриващи се интервали от време. Въпреки тези предимства, конкурентното изпълнение създава сериозни проблеми, когато различни нишки достъпват едни и същи ресурси.

Споделен ресурс е всяка стойност, обект, колекция, файл, база данни, мрежова връзка или друга част от системата, която може да бъде използвана от повече от една нишка. Ако ресурсът само се чете, рискът обикновено е по-малък. Проблемите възникват най-често, когато ресурсът е изменяем и няколко нишки могат едновременно да го променят. Тогава крайният резултат вече не зависи само от написания код, а и от реда, в който операционната система планира изпълнението на отделните нишки.

Race condition е един от основните проблеми в конкурентното програмиране. Той възниква, когато резултатът от програмата зависи от непредвидимия ред на изпълнение на нишките. Това означава, че една и съща програма може да даде различен резултат при различни стартирания, без кодът да е променен.

Причината за race condition е липсата на координация при достъп до споделено изменяемо състояние. Операции, които изглеждат елементарни, често не се изпълняват в една стъпка (не са атомарни). Например увеличаването на числова стойност с единица изглежда като една операция, но реално включва прочитане на старата стойност, изчисляване на новата стойност и записване обратно в паметта. Ако две нишки изпълнят тези стъпки едновременно, е възможно и двете да прочетат една и съща стара стойност и след това да запишат еднакъв резултат. Така едно от увеличенията се губи.

Race condition не е синтактична грешка и често не се проявява постоянно. Той може да се появи само при определено разпределение на времето между нишките. Именно затова този тип грешки са трудни за откриване, възпроизвеждане и поправяне.

Критична секция

Критична секция е част от програмния код, в която се достъпва споделен ресурс и която не трябва да се изпълнява едновременно от повече от една нишка. Ако няколко нишки могат едновременно да влязат в такава част от кода, съществува риск от неконсистентно състояние.

Критичната секция трябва да бъде защитена чрез синхронизационен механизъм. Целта е да се гарантира, че докато една нишка работи със споделения ресурс, останалите нишки ще изчакат. Така достъпът става последователен и предсказуем.

Lock е механизъм за взаимно изключване. Той гарантира, че само една нишка може да изпълнява дадена критична секция в определен момент. Когато една нишка влезе в заключен блок, останалите нишки, които се опитват да влязат в същия блок чрез същия заключващ обект, трябва да изчакат.

Lock се използва, когато трябва да се защити споделено изменяемо състояние. Той въвежда последователност при достъпа до ресурса и предотвратява race condition. От теоретична гледна точка lock осигурява атомарност на критичната секция, защото тя се изпълнява като неделима логическа операция спрямо другите нишки, които използват същото заключване.

Lock обаче не трябва да се използва безразборно. Ако критичната секция е твърде голяма или съдържа бавни операции, останалите нишки ще чакат по-дълго и производителността може да се влоши. Добрата практика е заключването да обхваща само най-малката необходима част от кода, в която реално се работи със споделения ресурс.

Deadlock е състояние на взаимно блокиране, при което две или повече нишки чакат една друга и никоя от тях не може да продължи. Това е едно от най-сериозните явления в конкурентното програмиране, защото програмата може да остане активна като процес, но реално да не извършва никакъв полезен напредък.

Deadlock често възниква, когато има повече от един заключван ресурс. Например една нишка може да заключи първия ресурс и след това да чака втори, докато друга нишка вече е заключила втория ресурс и чака първия. В този случай и двете нишки държат ресурс, който другата иска, и никоя не може да освободи своя ресурс, защото чака да получи другия.

Основната причина за deadlock е кръговото чакане. За да се предотврати, ресурсите трябва да се заключват в един и същ ред навсякъде в програмата. Ако всички нишки винаги заключват първо един и същ ресурс, а след това другия, не може да се образува цикъл на взаимно чакане.

Deadlock може да възникне не само при lock, но и при други механизми за синхронизация, включително semaphore. Всяка ситуация, в която нишка задържа ресурс и чака друг ресурс, може да създаде риск от взаимно блокиране.

Semaphore е синхронизационен механизъм, който ограничава броя на нишките или задачите, които могат едновременно да използват даден ресурс. За разлика от lock, който допуска само една нишка в критичната секция, semaphore може да позволи определен брой едновременни влизания.

Семафорът може да се разглежда като брояч. Когато нишка или задача иска да влезе, тя намалява брояча. Когато приключи работата си, тя го увеличава обратно. Ако броячът достигне нула, следващите нишки трябва да изчакат, докато някой освободи място.

Semaphore е подходящ, когато ресурсът може да бъде използван от повече от една нишка, но не от неограничен брой нишки. Например може да има ограничение за броя едновременни заявки към сървър, броя едновременни операции към файлова система или броя клиенти, които могат да използват дадена услуга.

В C# при асинхронно програмиране често се използва SemaphoreSlim, защото той поддържа асинхронно изчакване чрез WaitAsync. Това го прави по-подходящ от lock в ситуации, в които вътре в защитената логика има await. Lock не трябва да се комбинира с await, защото това може да доведе до неправилно поведение и блокиране.

Референтни стойности и споделено състояние

Референтните типове са особено важни при конкурентното програмиране, защото няколко нишки могат да държат препратка към един и същи обект в паметта. Това означава, че промяна, направена от една нишка, засяга същия обект, който може да бъде използван и от други нишки.

Ако обектът е изменяем, той се превръща в споделено изменяемо състояние. Това е една от най-опасните ситуации в конкурентното програмиране, защото всяка нишка може да промени състоянието в момент, в който друга нишка го чете или също го променя.

Проблемът не е само в самата промяна, а и във видимостта на тази промяна. В конкурентните системи не винаги е достатъчно една нишка да промени стойност. Трябва да има гаранция, че другите нишки ще видят тази промяна по предсказуем начин. Синхронизационните механизми като lock помагат не само за взаимно изключване, но и за подреждане и видимост на операциите.

Един от начините за намаляване на риска е използването на неизменяеми обекти. Ако даден обект не може да бъде променен след създаването си, тогава няколко нишки могат безопасно да го четат, без да има нужда от заключване. Затова неизменяемостта е важна стратегия при проектиране на конкурентни системи.

Starvation е състояние, при което дадена нишка не получава достъп до необходим ресурс за много дълго време или никога, въпреки че системата като цяло продължава да работи. Това се различава от deadlock, защото при starvation не всички нишки са блокирани. Някои нишки напредват, но една или повече нишки постоянно остават изпреварени.

Starvation е проблем на несправедливо разпределение. Той може да възникне, когато scheduler-ът, приоритетите или синхронизационният механизъм постоянно дават предимство на едни нишки пред други. В резултат някои нишки получават ресурсите многократно, докато други чакат прекалено дълго.

Този проблем е особено важен в системи, където всяка заявка или операция трябва да получи шанс за изпълнение. Ако няма гаранция за справедливост, системата може формално да работи, но да бъде некоректна спрямо част от участниците.

Livelock е състояние, при което нишките не са блокирани, но въпреки това не постигат реален напредък. Те продължават да реагират една на друга, променят поведението си и се опитват да избегнат конфликт, но тези реакции взаимно се неутрализират.

Разликата между deadlock и livelock е, че при deadlock нишките чакат пасивно, докато при livelock те остават активни. Въпреки активността, системата не стига до завършване на задачата. Това прави livelock по-труден за разпознаване, защото на пръв поглед изглежда, че програмата работи.

Thread safety е свойство на даден компонент, структура или клас да работи правилно при едновременен достъп от множество нишки. Един компонент е thread-safe, когато неговото вътрешно състояние остава коректно независимо от реда, в който нишките извикват неговите операции.

Thread safety не се постига само чрез добавяне на lock. Това е въпрос на цялостен дизайн. Трябва да се контролира кои данни са споделени, кога могат да се променят, кои операции трябва да бъдат атомарни и как се гарантира видимостта на промените между нишките.

Една структура може да бъде thread-safe чрез неизменяемост, чрез заключване, чрез атомарни операции или чрез специализирани конкурентни алгоритми. Изборът зависи от вида на данните, броя на едновременните операции и изискванията за производителност.

Дизайн на безопасни структури

Дизайнът на безопасни структури в конкурентното програмиране започва с ограничаване на споделеното изменяемо състояние. Колкото по-малко данни могат да бъдат променяни от няколко нишки, толкова по-лесно е системата да бъде коректна.

Вътрешното състояние на една структура трябва да бъде добре капсулирано. Това означава, че външният код не трябва да има неконтролиран достъп до променливите и вътрешните обекти. Ако вътрешен списък, речник или друг изменяем обект бъде върнат директно навън, друг код може да го промени без синхронизация и да наруши безопасността на структурата.

Критичните секции трябва да бъдат ясно определени и възможно най-кратки. Ако заключването обхваща твърде много код, се увеличава рискът от забавяне, блокиране и deadlock. Ако обаче заключването е твърде малко и не покрива цялата логическа операция, може да остане race condition.

Безопасната структура трябва да поддържа своите вътрешни правила за коректност. Това са условия, които винаги трябва да бъдат верни за обекта. Например балансът на акаунт не трябва да стане отрицателен, броят на елементите трябва да съответства на реалното съдържание, а вътрешните връзки между данните трябва да останат валидни. При конкурентен достъп синхронизацията трябва да гарантира, че тези правила не се нарушават дори временно по начин, който друга нишка може да наблюдава.

Обобщение

Конкурентното програмиране дава възможност няколко части от една програма да напредват едновременно, но това изисква внимателно управление на споделените ресурси. Когато няколко нишки достъпват едно и също изменяемо състояние, могат да възникнат race condition, deadlock, starvation и livelock.

Race condition води до непредвидим и често грешен резултат. Lock предпазва критичните секции, като допуска само една нишка в тях. Deadlock възниква, когато нишки се чакат взаимно и никоя не може да продължи. Semaphore ограничава броя едновременни достъпи до ресурс. Starvation означава, че някоя нишка не получава шанс за изпълнение, а livelock означава активност без реален напредък.

Основният принцип е, че всяко споделено изменяемо състояние трябва да бъде управлявано чрез ясен модел за синхронизация. Добре проектираната конкурентна система не разчита на случайния ред на изпълнение, а въвежда правила, които гарантират коректност, последователност и предсказуемост.